Python Environment Setup & Matplotlib Basics

Workshop 1 — Module 9

Prof.Asoc. Endri Raco

2025/26

Data Visualization for Data Scientists
UNYT Tirana

Module Roadmap

1. Installing Python, JupyterLab, and core packages
2. The Python data science stack: NumPy, pandas, matplotlib, seaborn
3. pandas data types and the DataFrame inspection workflow
4. Matplotlib architecture: Figure $\rightarrow$ Axes $\rightarrow$ Artists
5. The four fundamental chart types: scatter, histogram, bar, boxplot
6. pyplot vs Object-Oriented API

Learning Outcome

You will have a functioning Python environment and be able to produce all four fundamental chart types using matplotlib, preparing the ground for seaborn/plotnine in Workshop 2.

Python & JupyterLab Setup

Installation Checklist

Step 1: Install Python

```
# Option A: Miniconda (
    recommended)
# https://docs.conda.io/en/latest
    /
conda create -n dataviz python
    =3.11
conda activate dataviz

# Option B: pip + venv
python -m venv dataviz_env
source dataviz_env/bin/activate
```

Step 2: Install JupyterLab

```
pip install jupyterlab
jupyter lab  # opens browser
```

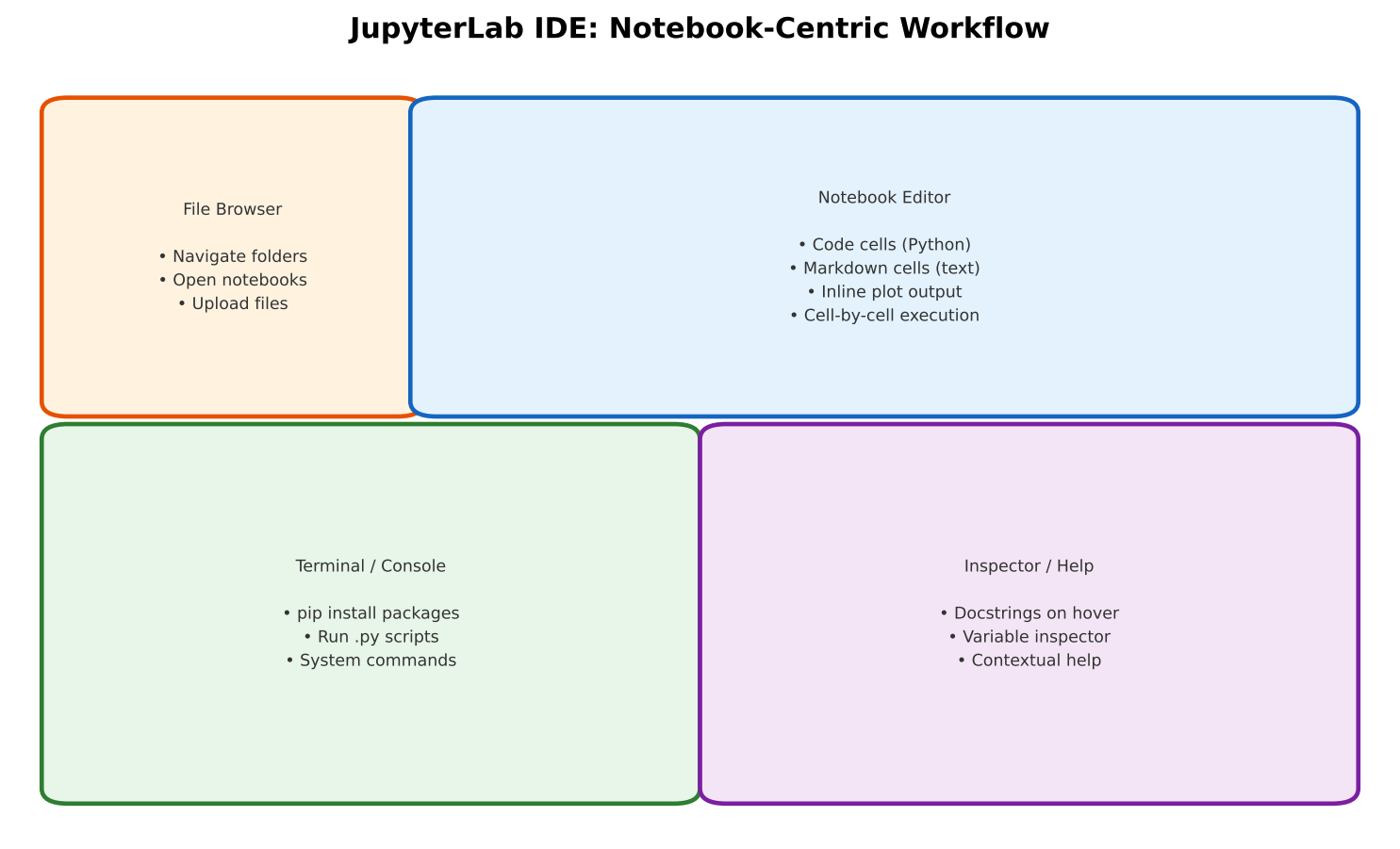
Step 3: Install core packages

```
pip install \
  numpy pandas \
  matplotlib seaborn \
  plotnine scipy \
  openpyxl  # for Excel files
```

Verify installation:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

print(f"numpy:  {np.__version__}"
    )
print(f"pandas: {pd.__version__}"
    )
print(f"mpl:    {plt.matplotlib
                    .__version__}
                        ")
print(f"sns:    {sns.__version__}
    ")
```

The JupyterLab IDE



JupyterLab Keyboard Shortcuts

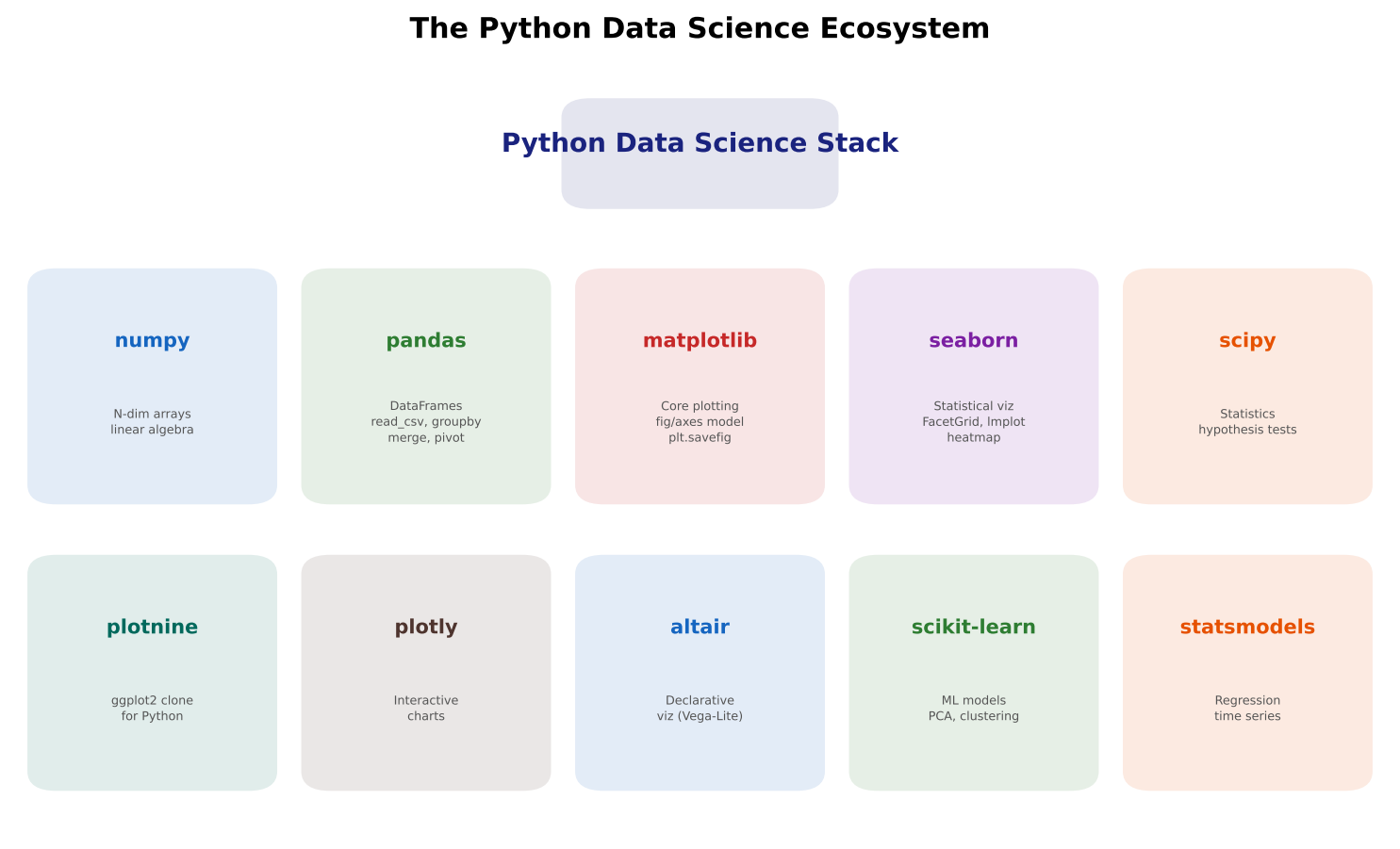
Action	Command Mode	Edit Mode
Run cell + next	Shift+Enter	Shift+Enter
Run cell (stay)	Ctrl+Enter	Ctrl+Enter
Insert cell above	A	—
Insert cell below	B	—
Delete cell	D, D	—
Switch to Markdown	M	—
Switch to Code	Y	—
Toggle line numbers	L	—

Pro-Tip

Press **Escape** to enter Command Mode (blue sidebar), **Enter** to enter Edit Mode (green sidebar). Most navigation happens in Command Mode; typing happens in Edit Mode.

The Python Data Science Stack

Python Ecosystem Overview



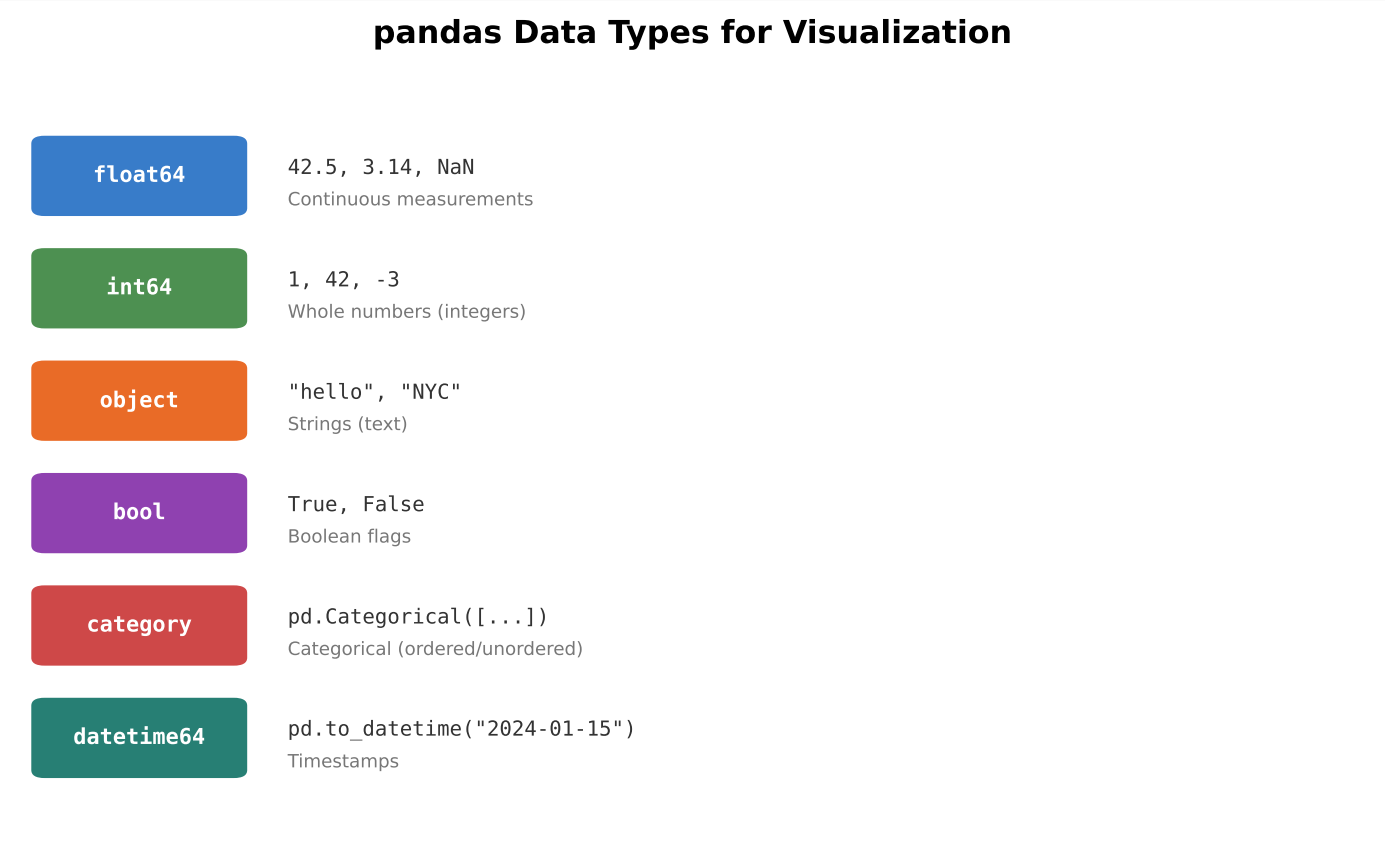
R ↔ Python Package Equivalents

Task	R	Python
Data frames	`tibble / data.frame`	`pandas.DataFrame`
CSV import	`readr::read_csv`	`pd.read_csv`
Data wrangling	`dplyr`	`pandas`
Static plots	`ggplot2`	`matplotlib / seaborn`
GoG clone	`ggplot2` (native)	`plotnine`
Interactive	`plotly / shiny`	`plotly / dash`
Statistics	built-in + `broom`	`scipy / statsmodels`
ML	`tidymodels`	`scikit-learn`

Data Insight
The ecosystems are deliberately parallel. Skills in one transfer to the other with mostly syntactic differences.

pandas Data Types

Data Types for Visualization



Pro-Tip

Use `df.dtypes` to check all column types and `df.info()` for a compact summary including non-null counts. Type mismatches (e.g., numbers parsed as `object`) are the number-one cause of plotting failures.

Loading and Inspecting Data

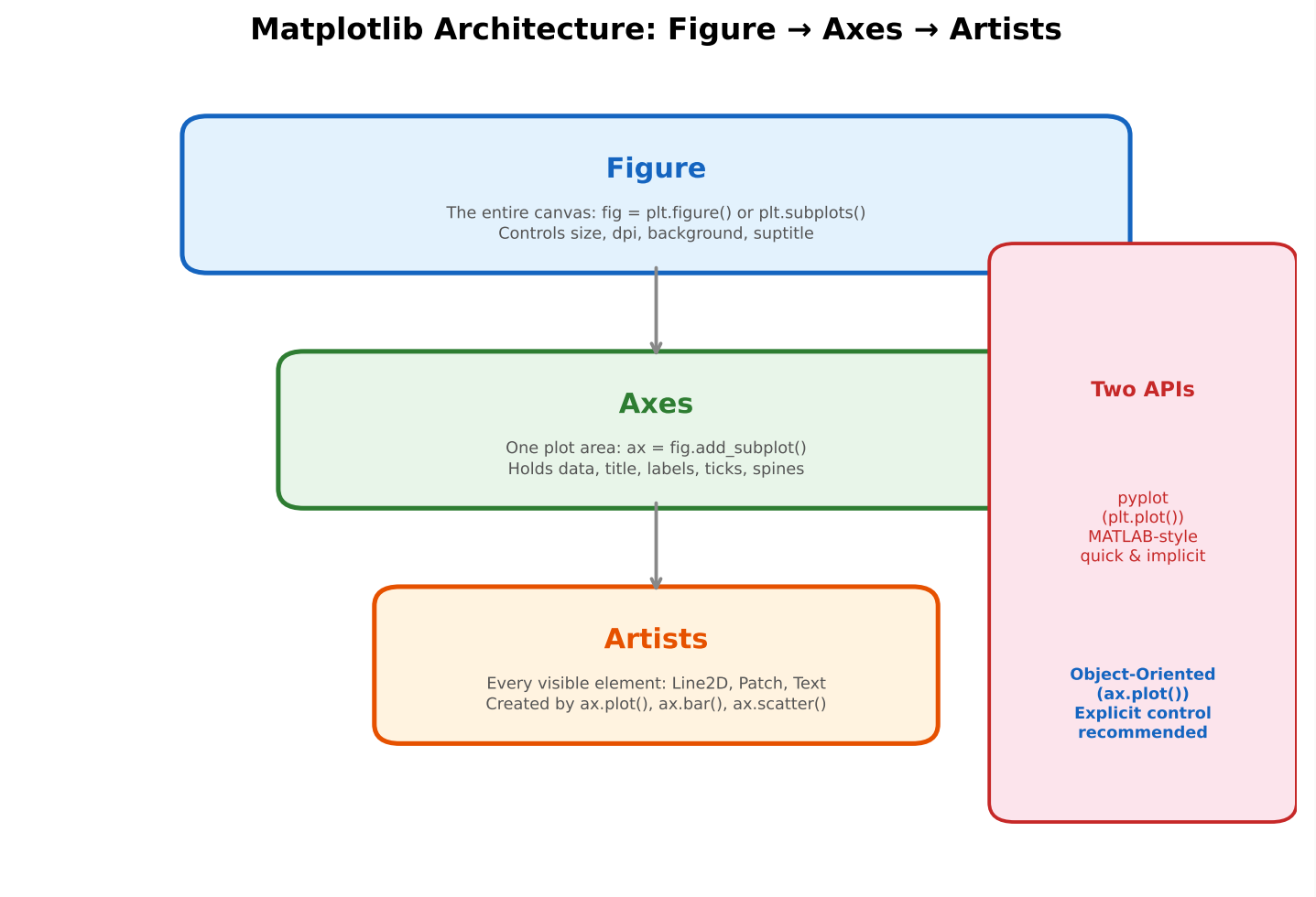
```python
import pandas as pd

# Read CSV
apps = pd.read_csv("googleplaystore.csv")

# Quick inspection (parallel to R's glimpse/str/summary)
print(apps.shape)          # (rows, cols)       like dim()
print(apps.dtypes)         # column types       like str()
print(apps.head())         # first 5 rows       like head()
print(apps.describe())     # summary stats      like summary()
print(apps.info())         # types + nulls      like glimpse()

# Missing values
print(apps.isna().sum())  # per-column NA count

# Useful for viz planning:
# - How many rows? (affects point density / alpha)
# - dtypes? (object = probably categorical or needs conversion)
# - Nulls? (need dropna() or fillna() before plotting)
```

Type Conversion Patterns

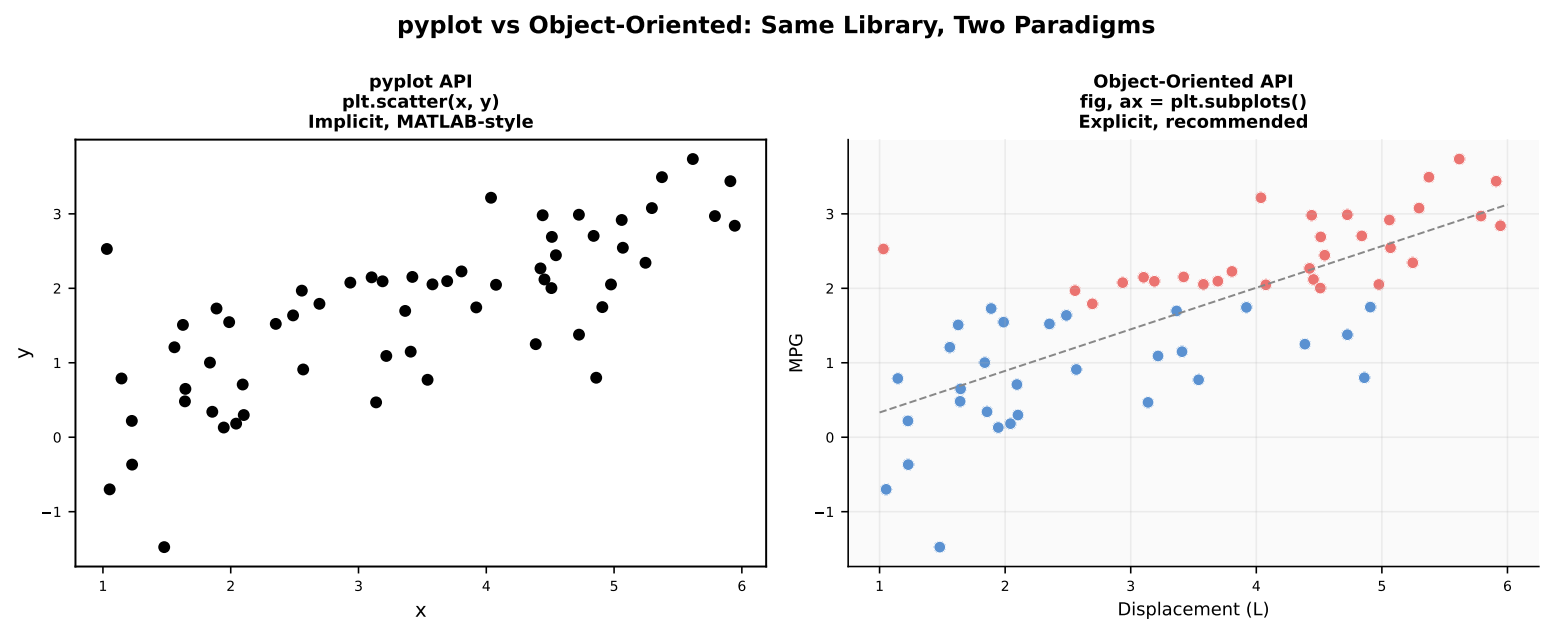
```python
# String to numeric (like as.numeric() in R)
apps["Reviews"] = pd.to_numeric(apps["Reviews"], errors="coerce")

# String to datetime (like as.Date() in R)
apps["Last Updated"] = pd.to_datetime(apps["Last Updated"],
                                      format="%B %d, %Y",
                                      errors="coerce")

# String to categorical (like factor() in R)
apps["Type"] = pd.Categorical(apps["Type"])

# Ordered category (like ordered factor)
rating_order = ["Everyone", "Teen", "Mature 17+"]
apps["Content Rating"] = pd.Categorical(
    apps["Content Rating"],
    categories=rating_order,
    ordered=True)

# Verify
print(apps.dtypes)
```

Matplotlib Architecture

Figure → Axes → Artists



pyplot vs Object-Oriented API



Data Insight

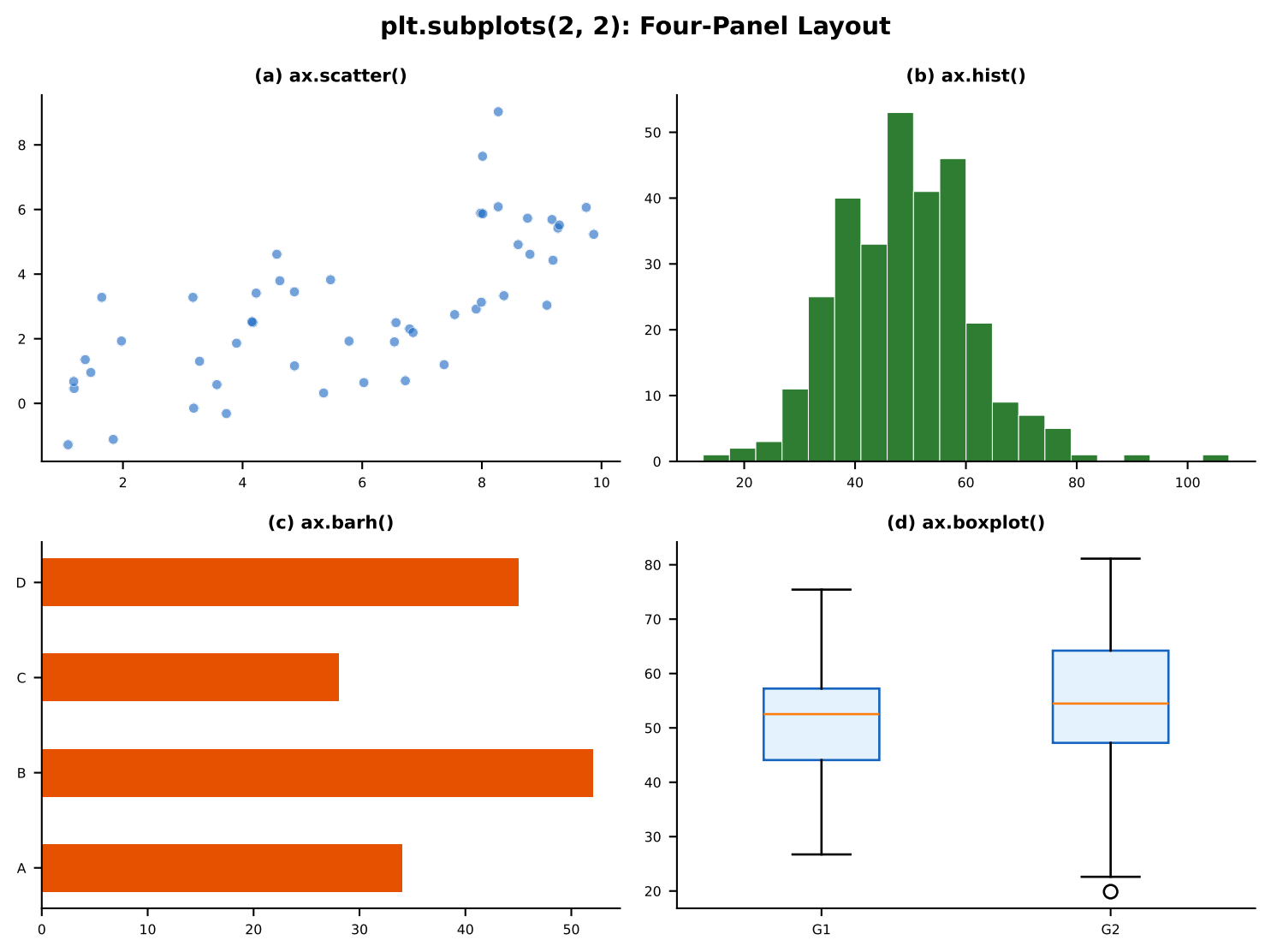
The **pyplot** API (`plt.plot()`) uses an implicit state machine modelled after MATLAB. The **OO** API (`ax.plot()`) gives explicit control over every figure element. This course uses the OO API exclusively — it is clearer, more composable, and the only viable approach for multi-panel layouts.

The OO Pattern: Your Template for Every Plot

```python
import matplotlib.pyplot as plt

# 1. Create figure and axes (ALWAYS start here)
fig, ax = plt.subplots(figsize=(7, 5))

# 2. Plot data onto the axes
ax.scatter(x, y, s=30, c="#1565C0", alpha=0.6)

# 3. Labels and title
ax.set_xlabel("X Variable", fontsize=10)
ax.set_ylabel("Y Variable", fontsize=10)
ax.set_title("My Chart Title", fontsize=12, fontweight="bold")

# 4. Tufte cleanup
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
ax.tick_params(labelsize=8)

# 5. Save (ALWAYS use bbox_inches="tight")
plt.tight_layout()
fig.savefig("chart.pdf", bbox_inches="tight")     # vector
fig.savefig("chart.png", dpi=300, bbox_inches="tight")  # raster
plt.close()  # free memory
```

Four Fundamental Chart Types

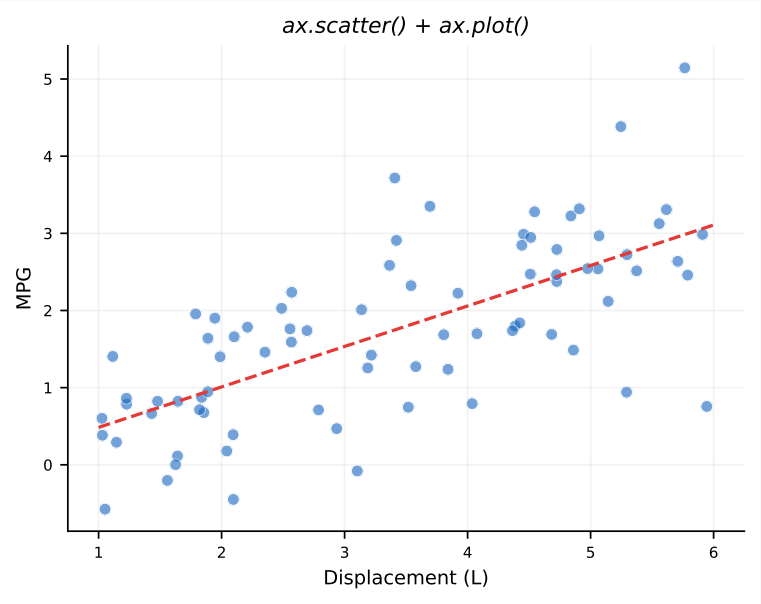
The Four Charts in matplotlib



Data Insight
These four methods — `ax.scatter()`, `ax.hist()`, `ax.barh()`,

Scatterplot: `ax.scatter()`

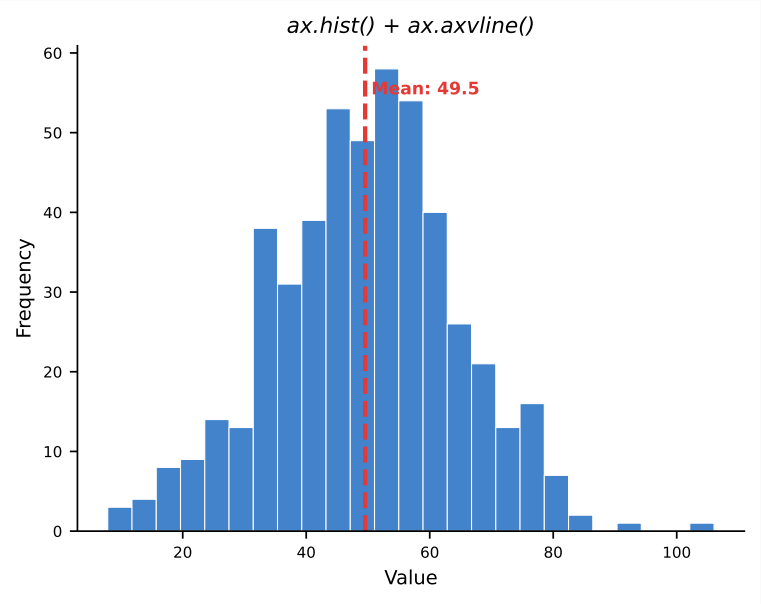
```
fig, ax = plt.subplots(figsize=(5,
    4))

ax.scatter(x, y,
    s=30,                  # marker size
    c="#1565C0",           # colour
    alpha=0.6,             #
        transparency
    edgecolors="white",
    linewidth=0.3)

# Add regression line
m, b = np.polyfit(x, y, 1)
xs = np.linspace(x.min(), x.max(),
    50)
ax.plot(xs, m*xs + b,
    color="#E53935", lw=1.5,
    linestyle="--")

ax.set_xlabel("Displacement (L)")
ax.set_ylabel("MPG")
ax.set_title("Scatter + OLS Fit")
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(
    False)
```



Key params: `s` = size (area in pts^2), `c` = colour (string or array), `alpha` = transparency [0–1], `edgecolors` = marker border.

Histogram: `ax.hist()`

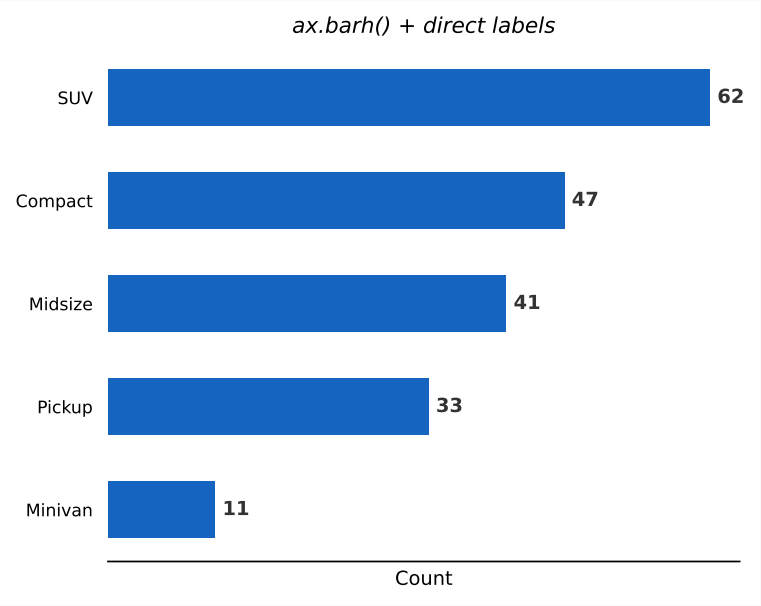
```
fig, ax = plt.subplots(figsize=(5,
    4))

ax.hist(data,
    bins=25,                # bin
        count
    color="#1565C0",
    edgecolor="white",
    linewidth=0.5,
    alpha=0.8)

# Mean reference line
mean_val = np.mean(data)
ax.axvline(x=mean_val,
    color="#E53935", lw=2,
    linestyle="--")
ax.text(mean_val + 1,
    ax.get_ylim()[1] * 0.9,
    f"Mean: {mean_val:.1f}",
    fontsize=8, color="#E53935",
    fontweight="bold")

ax.set_xlabel("Value")
ax.set_ylabel("Frequency")
```



Key params: `bins` = int or sequence, `density=True` for PDF, `histtype="step"` for unfilled, `cumulative=True` for CDF.

Bar Chart: `ax.barh()`

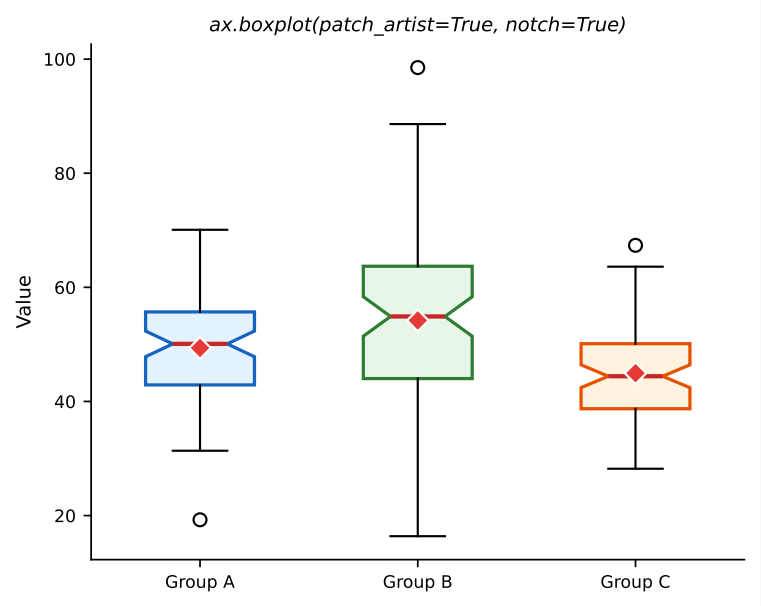
```python
fig, ax = plt.subplots(figsize=(5,
    4))

# Sort ascending for horizontal
idx = np.argsort(vals)
s_cats = [cats[i] for i in idx]
s_vals = [vals[i] for i in idx]

ax.barh(s_cats, s_vals,
    color="#1565C0",
    height=0.55,        # bar width
    edgecolor="none")

# Direct labels
for i, v in enumerate(s_vals):
    ax.text(v + 0.8, i, str(v),
        va="center", fontsize=9,
        fontweight="bold")

# Tufte cleanup
for sp in ["top","right","left"]:
    ax.spines[sp].set_visible(False
        )
ax.set_xticks([])
ax.tick_params(left=False)
```



Use `ax.barh()` (horizontal) by default: labels are readable without rotation. `ax.bar()` for vertical only when x-axis is temporal or ordinal.

Boxplot: `ax.boxplot()`

```
fig, ax = plt.subplots(figsize=(5,
    4))

bp = ax.boxplot(
    [group_a, group_b, group_c],
    labels=["A", "B", "C"],
    patch_artist=True,  # fill
        boxes
    widths=0.5,
    notch=True)         # CI on
        median

# Customise box colours
fills = ["#E3F2FD","#E8F5E9","#
    FFF3E0"]
edges = ["#1565C0","#2E7D32","#
    E65100"]
for patch, fc, ec in zip(
    bp["boxes"], fills, edges):
    patch.set_facecolor(fc)
    patch.set_edgecolor(ec)

# Overlay mean points
means = [np.mean(d) for d in data]
ax.scatter(range(1, 4), means,
```



Key params: `patch_artist=True` enables fill colour; `notch=True` adds median CI; `whis=1.5` (default) controls whisker extent; use `vert=False` for horizontal.

Multi-Panel Layouts

plt.subplots(rows, cols)

```
# Create 2x2 grid (equivalent to R's par(mfrow = c(2,2)))
fig, axes = plt.subplots(2, 2, figsize=(8, 6))

# Access each axes by index
axes[0, 0].scatter(x, y, s=20, c="#1565C0")
axes[0, 0].set_title("(a) Scatter")

axes[0, 1].hist(data, bins=20, color="#2E7D32")
axes[0, 1].set_title("(b) Histogram")

axes[1, 0].barh(cats, vals, color="#E65100", height=0.5)
axes[1, 0].set_title("(c) Bar Chart")

axes[1, 1].boxplot(groups, patch_artist=True)
axes[1, 1].set_title("(d) Boxplot")

# Tufte cleanup on all panels
for ax in axes.flatten():
    ax.spines["top"].set_visible(False)
    ax.spines["right"].set_visible(False)
    ax.tick_params(labelsize=7)

plt.tight_layout()  # prevent label overlap
fig.savefig("multipanel.pdf", bbox_inches="tight")
```

Advanced: GridSpec for Unequal Panels

```
from matplotlib.gridspec import GridSpec

fig = plt.figure(figsize=(10, 6))
gs = GridSpec(2, 3, figure=fig,
    hspace=0.4, wspace=0.3,
    height_ratios=[1, 1.5])

# Top row: three equal panels
ax1 = fig.add_subplot(gs[0, 0])  # row 0, col 0
ax2 = fig.add_subplot(gs[0, 1])  # row 0, col 1
ax3 = fig.add_subplot(gs[0, 2])  # row 0, col 2

# Bottom row: one full-width panel
ax4 = fig.add_subplot(gs[1, :])  # row 1, ALL cols

# Shared y-axis: sharey=ax1
ax5 = fig.add_subplot(gs[0, 2], sharey=ax1)
```

Pro-Tip

GridSpec is the Python equivalent of R's patchwork package. Use it for any layout that is not a regular grid. Slice notation (gs[1, :]) spans multiple columns.

Synthesis

R vs Python: Side-by-Side Cheat Sheet

Operation	R (base)	Python (matplotlib)
Scatter	`plot(x, y)`	`ax.scatter(x, y)`
Histogram	`hist(x)`	`ax.hist(x)`
Bar chart	`barplot(table(x))`	`ax.barh(cats, vals)`
Boxplot	`boxplot(y ~ g)`	`ax.boxplot([g1, g2])`
Multi-panel	`par(mfrow=c(2,2))`	`plt.subplots(2, 2)`
Save plot	`png(); ...; dev.off()`	`fig.savefig("f.png")`
Reg. line	`abline(lm(y~x))`	`ax.plot(xs, m*xs+b)`
Ref. line	`abline(h=v)`	`ax.axhline(y=v)`

Key Takeaways

1. **Miniconda + JupyterLab** is the recommended Python setup. Learn Shift+Enter and Command/Edit mode switching.
2. **pandas** is the Python equivalent of tibble + dplyr. Use `df.dtypes` and `df.info()` before plotting.
3. Matplotlib has **three layers**: Figure (canvas), Axes (plot area), Artists (data marks). Always think in this hierarchy.
4. Use the **OO API** (`fig, ax = plt.subplots()`) exclusively — it is explicit, composable, and scalable.
5. Master four methods: `ax.scatter()`, `ax.hist()`, `ax.barh()`, `ax.boxplot()`.
6. `plt.subplots(r, c)` for regular grids; `GridSpec` for unequal panels.

Essential Readings for This Module

- VanderPlas, J. (2016). *Python Data Science Handbook*, Chapter 4 (Visualization with Matplotlib). `https://jakevdp.github.io/PythonDataScienceHandbook/`
- McKinney, W. (2022). *Python for Data Analysis*, 3rd ed., Chapters 4–5 (NumPy, pandas) and Chapter 9 (Plotting).
- Matplotlib official tutorials: `https://matplotlib.org/stable/tutorials/`

Next Module: Comparative Lab — First Plots in R vs Python (W01-M10)